

FACULTAD DE ECONOMÍA, EMPRESA Y NEGOCIOS.
Metodologías de Desarrollo de Software 0

DOCUMENTO DE VERIFICACIÓN Y VALIDACIÓN DE SOFTWARE (QA)

Proyecto Integrador SumaControl — Plan de Pruebas Funcionales, Unitarias e Integrales

Catedrático/a:
Lcda. Ana Lissette Girón de Bermúdez

Presentado por:
Victor Eduardo Alonzo López
Samuel Rafael Martínez Ramírez
Gonzalo Ricardo Guzmán Sosa

Unidad Didáctica 3 — Tema 6: Verificación y Validación del Software

Antiguo Cuscatlán, Noviembre de 2026

1. INFORMACIÓN DE CONTROL Y GOBERNANZA DEL PROYECTO (IEEE/ISO 29148)

El presente documento constituye el Plan de Verificación y Validación de Software (QA) de la Versión Beta Funcional del proyecto SumaControl. Recoge la estrategia de pruebas en sus tres niveles —unitario, funcional y de integración—, la matriz de trazabilidad entre las reglas de negocio y la API REST, y la evidencia de gobernanza del repositorio. Todo el contenido técnico corresponde a la implementación real del equipo: los endpoints, los nombres de los servicios y los valores de configuración citados fueron verificados contra el código fuente de la rama de integración, no tomados del ejemplo resuelto de la plantilla.

1.1 Hoja de control de entregables e identificación del equipo

Empresa / Institución:	Centro Logístico SumaControl / UJMD-FEEN	**Código Proyecto:**	GTD-MDS0-PON0-S12
Proyecto Integrador:	Sistema de Gestión Logística de Recepción de Proveedores y Desembarque	**Fase Evaluada:**	Fase III — Versión Beta Funcional
Asignatura Gobierno:	Metodologías de Desarrollo de Software 0 (MDS 0)	**Hito / Periodo:**	Actualización para el Plan de Implementación (noviembre 2026)
Equipo Scrum / Grupo:	Equipo Scrum N.º 1 (Sección 1-1)	**Product Owner (PO):**	Victor Eduardo Alonzo López
Scrum Master (SM):	Samuel Rafael Martínez Ramírez	**Dev Team:**	Gonzalo Ricardo Guzmán Sosa
Versión / Línea Base:	v2.1.0-beta (Sprint 5 cerrado)	**Estado de Aprobación:**	Entregado para evaluación del Segundo Parcial

1.2 Registro de cambios y versionamiento de pruebas (SCM)

Versión	Causa del cambio / incremento	Responsable	Rama / etiqueta	Fecha
v1.0.0	Línea base del backend: autenticación JWT, RBAC por rol y modelo de usuario (Sprint 0).	Gonzalo Guzmán	rama: sprint0-auth	Sep/2026
v1.5.0	Gestión de proveedores, pedidos y ventanas horarias; registro de llegada y clasificación de puntualidad (Sprints 1 y 2).	Victor Alonzo	rama: feature/sprint2-llegadas	Sep/2026
v1.8.0	Ciclo de descarga en gateways, promoción automática de cola y bitácora de eventos (Sprint 3).	Gonzalo Guzmán	rama: feature/sprint3-gateways	Oct/2026
v2.0.0-beta	Tablero de indicadores (KPIs) e incorporación de la suite de pruebas unitarias Jest: 48 casos sobre las reglas RN-01 a RN-04, RN-07, RN-08 y RN-14 (Sprint 4).	Equipo Scrum QA	rama: feature/HU-03-pruebas-qa tag: v2.0.0-beta	Oct/2026
v2.1.0-beta	Autenticación en dos pasos con TOTP, códigos de recuperación, límite de intentos por token temporal y administración de usuarios. Suma 30 pruebas unitarias (PU-19 a PU-25) y las cuatro pruebas integrales End-to-End PI-01 a PI-04 (Sprint 5).	Equipo Scrum QA	rama: sprint5-samuel tag: v2.1.0-beta	Nov/2026

Nota sobre el versionamiento: la numeración de versiones de este registro refleja los incrementos funcionales reales del repositorio y se ancla a ramas existentes. El repositorio no tenía etiquetas antes de esta entrega: la etiqueta v2.1.0-beta se crea como parte del cierre de liberación documentado en la sección 6.2.

2. ESTRATEGIA GENERAL DE VERIFICACIÓN Y VALIDACIÓN (QA)

La estrategia de pruebas de la Versión Beta Funcional abarca tres niveles complementarios. Las pruebas unitarias (sección 4) evalúan de forma aislada las funciones puras que concentran la lógica de negocio del backend, sin levantar la base de datos ni el servidor HTTP. Las pruebas funcionales y de aceptación (sección 3) recorren los doce casos de uso operativos del centro de distribución desde la interfaz hasta la persistencia. Las pruebas integrales (sección 5) verifican la interacción conjunta del frontend Next.js, la API Express, MongoDB y el control de acceso por roles.

Esta separación no es decorativa: responde a dónde vive cada regla en la arquitectura. El equipo escribió los servicios del backend como funciones puras que reciben la configuración por parámetro en lugar de ir a buscarla a la base, y esa decisión de diseño es la que permite probar las reglas de negocio sin infraestructura. Las reglas que sí dependen del estado compartido —ocupación de bahías, promoción de cola— se validan en los niveles funcional e integral, donde esa dependencia es parte de lo que se está probando.

2.1 Matriz de trazabilidad: Regla de Negocio ↔ API ↔ Caso de Prueba

La siguiente matriz consolida el catálogo de reglas de negocio RN-01 a RN-17 y lo vincula de forma bidireccional con la Historia de Usuario que lo origina, el Caso de Uso UML que lo modela, el endpoint o componente que lo implementa y los casos de prueba que lo verifican.

Regla de Negocio	HU / CU	Endpoint o componente real	Casos de prueba	Criterio de éxito esperado
RN-01 Horario operativo	HU-001 / CU-01	`POST /api/pedidos` `services/ventanaHoraria.ts`	CP-11 / PU-10	La ventana de la cita cae completa dentro del horario configurable (07:00 a 17:00) y en un día hábil; si no, se rechaza con 400.
RN-02 No solapamiento	HU-001 / CU-01	`POST /api/pedidos` `services/ventanaHoraria.ts`	CP-11 / PU-11, PU-12	Dos citas no comparten franja. Dos turnos consecutivos (09:00-10:00 y 10:00-11:00) sí son válidos: la comparación es estricta.
RN-03 Correlativo de pedido	HU-001 / CU-01	`POST /api/pedidos` `services/numeroPedido.ts`	CP-12 / PU-15 a PU-18	Autogenera PED-EQUI04-NNNNNNNNN con nueve dígitos. El correlativo de un pedido cancelado no se reutiliza.
RN-04 Tolerancia de puntualidad	HU-002 / CU-02	`POST /api/llegadas` `services/puntualidad.ts`	CP-01 / CP-03 PU-01 a PU-05	Clasifica en ANTICIPADO, A TIEMPO, TARDÍO o AUSENTE según los tres umbrales configurables (15 / 15 / 60 min). El minuto exacto del margen cuenta a favor del proveedor.
RN-05 Timestamp del servidor	HU-002 / CU-02	`POST /api/llegadas` `controllers/ llegadaController.ts`	CP-01	La hora de arribo la fija el servidor y no el cliente, para garantizar la trazabilidad del registro.
RN-06 Marcado de ausencia	HU-002 / CU-02	`services/ controlAusencias.ts`	CP-04	Vencido el límite de ausencia sin llegada registrada, el pedido pasa a AUSENTE y el hecho queda en la bitácora.
RN-07 Compatibilidad de carga	HU-003 / CU-03	`POST /api/descargas/iniciar` `models/Gateway.ts`	CP-01 / CP-06 PU-07, PU-08	Los gateways 1 a 4 admiten únicamente carga general. Un pedido de otro tipo se rechaza con 400.
RN-08 Exclusividad Gateway 5	HU-003 / CU-03	`POST /api/descargas/iniciar` `models/Gateway.ts`	CP-05 / CP-06 PU-06, PU-07	El Gateway 5 es la única bahía de carga de construcción y nunca se asigna a carga general, aunque sea la única libre.
RN-09 Ocupación atómica	HU-003 / CU-03	`POST /api/descargas/iniciar` `services/descargas.ts`	CP-07	Una bahía solo se ocupa si está LIBRE. La actualización es condicional, así dos peticiones simultáneas nunca ocupan la misma bahía.
RN-10	HU-003 /	`POST`	CP-02	Si no hay bahía compatible libre, el pedido permanece

Regla de Negocio	HU / CU	Endpoint o componente real	Casos de prueba	Criterio de éxito esperado
Permanencia en cola	CU-03	`/api/descargas/iniciar` `services/descargas.ts`		en espera y no se fuerza una asignación incompatible.
RN-11 Liberación y promoción	HU-003 / CU-03	`POST /api/descargas/finalizar` `services/descargas.ts`	CP-07 / CP-08	Al finalizar, la bahía vuelve a LIBRE y el sistema promueve automáticamente al siguiente pedido compatible de la cola.
RN-12 Orden de atención	HU-003 / CU-03	`services/descargas.ts` `(promoverPrimeroDeLaCola)`	CP-10	La cola se atiende por hora de llegada real: pasa quien lleva más tiempo esperando en el patio. Ver la nota técnica de la sección 5.3.
RN-13 Bahía fuera de servicio	HU-003 / CU-03	`PUT /api/gateways/:id` `controllers/ gatewayController.ts`	CP-09	Un gateway FUERA DE SERVICIO no recibe asignaciones, ni manuales ni por promoción automática.
RN-14 Reprogramación con historial	HU-001 / CU-01	`PUT /api/pedidos/:id/reprogramar` `services/ventanaHoraria.ts`	CP-11 PU-13, PU-14	La reprogramación valida horario y solapamiento, deja constancia de la ventana anterior y la nueva en el evento CITA_REPROGRAMADA, y ofrece hasta tres franjas alternativas saltando los días no hábiles.
RN-15 Decisión del paso de login	HU-004 / CU-04	`POST /api/auth/login` `services/login.ts`	PU-19, PU-20	El login resuelve en uno de tres pasos: acceso directo, configurar el segundo factor o verificarlo. Nunca entrega acceso directo si el segundo factor es obligatorio o ya está configurado.
RN-16 Formato de códigos	HU-004 / CU-04	`POST /api/auth/mfa/verify-login` `services/mfa.ts`	PU-21 a PU-24	El código del segundo factor tiene exactamente seis dígitos. Los códigos de recuperación usan el formato XXXXX-XXXXX, no se repiten, evitan caracteres confundibles y se almacenan hasheados.
RN-17 Límite de intentos	HU-004 / CU-04	`services/intentosMfa.ts`	PU-25	El conteo de intentos fallidos es por token temporal y no por usuario. Corta después del quinto código incorrecto; un acierto lo reinicia.

📌 REGLA DE ORO DE QA Y COBERTURA: toda prueba consignada en este documento se ejecuta contra la implementación real del repositorio del equipo, no contra un prototipo. Las pruebas unitarias corren con Jest sobre el código de la rama feature/HU-03-pruebas-qa y su salida completa se adjunta en el Anexo 6.1; las pruebas funcionales e integrales se ejecutan sobre la API Express desplegada en entorno local contra MongoDB, con el banco automatizado de backend/src/pruebas/. Ningún resultado de este documento se declara PASS sin su evidencia correspondiente: cada «Resultado obtenido» transcribe lo que la corrida imprimió, y los hallazgos que la ejecución dejó al descubierto se consignan en la sección 3.2 en lugar de omitirse.

Origen del catálogo de reglas: el documento de la Semana 7 formaliza explícitamente RN-04, RN-08 y RN-14 como flujos de extensión de los Casos de Uso. El catálogo completo RN-01 a RN-17 que aparece en esta matriz se consolidó en la Semana 12 a partir de dos fuentes propias del equipo: los criterios de aceptación en sintaxis Gherkin de las Historias de Usuario refinadas en la Semana 8 y las validaciones efectivamente implementadas en el backend. Las reglas RN-15 a RN-17 se incorporan en esta revisión y corresponden al incremento del Sprint 5: la decisión del paso de login, el formato de los códigos del segundo factor y el límite de intentos. No se incorporaron reglas que el sistema no aplique.

3. PLAN DE PRUEBAS FUNCIONALES (SISTEMA Y ACEPTACIÓN — CP-01 A CP-12)

A continuación se especifican las fichas detalladas de los doce casos de prueba funcionales del Proyecto Integrador SumaControl. Cada ficha declara la regla de negocio que verifica, de modo que la cobertura sobre el catálogo RN-01 a RN-17 pueda auditarse desde la matriz de la sección 2.1 en ambos sentidos.

Caso de Prueba:	**CP-01 — Proveedor puntual con gateway general libre**
Regla(s) verificada(s):	RN-04, RN-05, RN-07
Descripción:	Verificar que al registrar la llegada de un proveedor dentro de la ventana de tolerancia, el sistema lo clasifique como A TIEMPO y permita iniciar la descarga en una bahía de carga general.
Prerrequisitos:	1. Pedido en estado PROGRAMADO con ventana que inicia a las 09:00. 2. Tipo de producto "general". 3. Gateway 1 en estado LIBRE. 4. Parámetros sembrados con npm run seed:parametros.
Pasos a ejecutar:	1. El Operador ingresa a /operador/llegadas y selecciona el pedido. 2. Confirma la llegada a las 09:12 (12 minutos de diferencia). 3. El Coordinador inicia la descarga del pedido en el Gateway 1.
Resultado esperado:	1. POST /api/llegadas responde 200 y guarda puntualidad = "A TIEMPO". 2. La hora registrada es la del servidor, no la del navegador (RN-05). 3. POST /api/descargas/iniciar responde 201; el Gateway 1 pasa a OCUPADO y el pedido a DESCARGANDO. 4. Se registran los eventos GATEWAY_ASIGNADO y DESCARGA_INICIADA.
Resultado obtenido:	La API respondió 200 y clasificó la llegada como A TIEMPO (−5 min de diferencia). El pedido guardó la hora de arribo fijada por el servidor (2026-10-20T06:32:29.711Z). El inicio de descarga respondió 201: el Gateway 1 quedó OCUPADO y el pedido en DESCARGANDO.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-02 — Proveedor puntual sin bahía general disponible**
Regla(s) verificada(s):	RN-10
Descripción:	Validar que si todas las bahías de carga general están ocupadas, el pedido permanezca en espera sin que el sistema fuerce una asignación incompatible.
Prerrequisitos:	1. Gateways 1, 2, 3 y 4 en estado OCUPADO. 2. Gateway 5 en estado LIBRE. 3. Pedido de tipo "general" con llegada ya registrada.
Pasos a ejecutar:	1. Se registra la llegada del pedido en garita. 2. Se intenta iniciar la descarga en cada uno de los gateways disponibles.
Resultado esperado:	1. La clasificación de puntualidad se asigna con normalidad. 2. Ningún intento de asignación prospera: los gateways 1 a 4 responden 400 por no estar LIBRES y el Gateway 5 responde 400 por incompatibilidad de carga. 3. El pedido conserva su estado elegible y queda esperando bahía.
Resultado obtenido:	Los cuatro gateways generales rechazaron la asignación con 400 por no estar LIBRES. El Gateway 5 la rechazó con «El gateway 5 solo admite carga "construcción"». El pedido quedó en estado EN COLA, esperando bahía compatible, sin que el sistema forzara una asignación incompatible.

Estado:	✔ **PASS**

Caso de Prueba:	**CP-03 — Proveedor tardío que excede la ventana de tolerancia**
Regla(s) verificada(s):	RN-04
Descripción:	Comprobar el cálculo automático de puntualidad cuando el arribo supera el margen de tolerancia configurado pero no alcanza el límite de ausencia.
Prerrequisitos:	1. Pedido con ventana que inicia a las 08:00. 2. Tolerancia de tardanza configurada en 15 minutos. 3. Límite de ausencia configurado en 60 minutos.
Pasos a ejecutar:	1. El proveedor se presenta a las 08:25 (25 minutos de diferencia). 2. El Operador confirma la llegada en la pantalla de garita.
Resultado esperado:	1. El backend calcula 25 minutos de diferencia contra el inicio de la ventana. 2. La puntualidad queda registrada como "TARDÍO". 3. El pedido sigue siendo elegible para descarga: llegar tarde no lo excluye.
Resultado obtenido:	La API respondió 200 y clasificó la llegada como TARDÍO, calculando 25 minutos de diferencia contra el inicio de la ventana. La puntualidad quedó persistida en el pedido y este siguió siendo elegible para descarga.
Estado:	✔ **PASS**

Caso de Prueba:	**CP-04 — Proveedor ausente que no se presenta en toda la ventana**
Regla(s) verificada(s):	RN-06
Descripción:	Verificar que al superarse el límite de ausencia sin registro de arribo, el sistema marque el pedido como AUSENTE de forma automática y libere la franja.
Prerrequisitos:	1. Pedido PROGRAMADO con ventana que inicia a las 09:00. 2. Límite de ausencia de 60 minutos (vence 10:00). 3. Ninguna llegada registrada para ese pedido.
Pasos a ejecutar:	1. Se deja transcurrir la hora del servidor más allá del límite. 2. Se ejecuta la rutina de control de ausencias. 3. Se consulta el pedido y la bitácora de eventos.
Resultado esperado:	1. El estado del pedido pasa automáticamente a AUSENTE. 2. La franja horaria deja de contar como ocupada para el control de solapamiento. 3. El evento queda asentado en GET /api/eventos con usuario, fecha y detalle.
Resultado obtenido:	El control de ausencias respondió 200 y actualizó el pedido a AUSENTE sin intervención manual. La bitácora quedó consultable en GET /api/eventos.
Estado:	✔ **PASS**

Caso de Prueba:	**CP-05 — Producto de construcción asignado al Gateway 5**
Regla(s) verificada(s):	RN-08
Descripción:	Validar que un pedido de tipo "construcción" se asigne exclusivamente a la bahía 5, que es la única habilitada para ese tipo de carga.

Prerrequisitos:	1. Pedido con tipoProducto = "construcción" y llegada registrada. 2. Gateway 5 en estado LIBRE. 3. Gateways 1 a 4 en estado LIBRE.
Pasos a ejecutar:	1. Se intenta iniciar la descarga del pedido en el Gateway 2. 2. Se inicia la descarga del mismo pedido en el Gateway 5.
Resultado esperado:	1. El intento sobre el Gateway 2 se rechaza con 400 y el mensaje "El gateway 2 solo admite carga general". 2. El inicio en el Gateway 5 responde 201: la bahía pasa a OCUPADO y el pedido a DESCARGANDO.
Resultado obtenido:	El intento sobre el Gateway 2 se rechazó con «El gateway 2 solo admite carga "general"». El inicio en el Gateway 5 respondió 201: la bahía quedó OCUPADA y el pedido en DESCARGANDO.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-06 — Carga general rechazada en el Gateway 5 ocupado el resto**
Regla(s) verificada(s):	RN-07, RN-08
Descripción:	Verificar que la exclusividad del Gateway 5 se respete incluso cuando es la única bahía libre del centro de distribución: el sistema prefiere la espera antes que romper la regla.
Prerrequisitos:	1. Gateways 1, 2, 3 y 4 en estado OCUPADO. 2. Gateway 5 en estado LIBRE. 3. Pedido de tipo "general" con llegada registrada.
Pasos a ejecutar:	1. Se intenta iniciar la descarga del pedido general en el Gateway 5. 2. Se verifica el estado del Gateway 5 después del intento.
Resultado esperado:	1. La operación se rechaza con 400: "El gateway 5 solo admite carga construcción". 2. El Gateway 5 permanece LIBRE; el rechazo no lo deja ocupado por error. 3. El pedido conserva su estado y sigue esperando una bahía compatible.
Resultado obtenido:	La operación se rechazó con «El gateway 5 solo admite carga "construcción"» aun siendo la única bahía libre del centro. El Gateway 5 permaneció LIBRE tras el rechazo y el pedido conservó su estado de espera.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-07 — Finalización de descarga en bahía general y promoción de cola**
Regla(s) verificada(s):	RN-09, RN-11, RN-12
Descripción:	Validar que al finalizar una descarga la bahía se libere y el sistema promueva automáticamente al siguiente pedido compatible que lleva más tiempo esperando.
Prerrequisitos:	1. Descarga activa en el Gateway 1 con el pedido A. 2. Pedidos B y C de tipo "general", ambos con llegada registrada. 3. El pedido B llegó antes que el C.
Pasos a ejecutar:	1. El Operador finaliza la descarga del pedido A en el Gateway 1. 2. Se consulta el estado del Gateway 1 y de los pedidos B y C.
Resultado esperado:	1. El pedido A pasa a FINALIZADO y se calcula la duración de la descarga. 2. El Gateway 1 se libera y de inmediato se reasigna al pedido B, no al C. 3. Se registran los eventos de finalización y de la nueva asignación automática.

Resultado obtenido:	La finalización respondió 200 y el pedido en descarga quedó FINALIZADO. La bahía liberada se reasignó automáticamente al pedido cuya llegada era más antigua, no al más reciente.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-08 — Finalización en Gateway 5 con promoción filtrada por tipo**
Regla(s) verificada(s):	RN-08, RN-11, RN-12
Descripción:	Verificar que al liberarse el Gateway 5 la promoción automática considere únicamente pedidos de construcción, aunque haya pedidos generales que llevan más tiempo esperando.
Prerrequisitos:	1. Descarga activa en el Gateway 5. 2. En espera: un pedido general que llegó a las 08:00 y uno de construcción que llegó a las 10:00.
Pasos a ejecutar:	1. Se finaliza la descarga activa en el Gateway 5. 2. Se consulta a qué pedido se reasignó la bahía.
Resultado esperado:	1. El Gateway 5 se reasigna al pedido de construcción pese a haber llegado después. 2. El pedido general permanece en espera: la antigüedad no sobrescribe a RN-08.
Resultado obtenido:	Al liberarse el Gateway 5, la promoción automática seleccionó el pedido de construcción pese a que el pedido general llevaba más tiempo esperando. La antigüedad en cola no sobrescribió la exclusividad de RN-08.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-09 — Bahía fuera de servicio excluida de toda asignación**
Regla(s) verificada(s):	RN-13
Descripción:	Validar que un gateway marcado como FUERA DE SERVICIO no reciba descargas, ni por asignación manual ni por promoción automática de la cola.
Prerrequisitos:	1. Gateway 3 configurado en estado FUERA DE SERVICIO. 2. Pedido de tipo "general" con llegada registrada y en espera.
Pasos a ejecutar:	1. Se intenta iniciar manualmente una descarga en el Gateway 3. 2. Se finaliza una descarga en otra bahía para disparar la promoción automática.
Resultado esperado:	1. La asignación manual al Gateway 3 se rechaza. 2. La promoción automática no considera al Gateway 3 como destino posible. 3. El estado del Gateway 3 permanece sin cambios.
Resultado obtenido:	Con las otras cuatro bahías LIBRES, la asignación al Gateway 3 inhabilitado se rechazó con 400 y la bahía conservó su estado. El control del experimento confirmó la causa: el mismo pedido, sin modificar, fue aceptado con 201 en el Gateway 1. Ver la observación de QA en la sección 3.2.
Estado:	✔️ **PASS**

Caso de Prueba:	**CP-10 — Orden de atención de la cola por antigüedad de arribo**
Regla(s) verificada(s):	RN-12

Descripción:	Comprobar que la cola de espera se atienda por hora de llegada real y no por la hora programada de la cita: pasa primero quien lleva más tiempo en el patio.
Prerrequisitos:	1. Tres pedidos de tipo "general" en espera. 2. P1 con cita 09:00 y llegada 09:40; P2 con cita 10:00 y llegada 09:20; P3 con cita 08:00 y llegada 10:10. 3. Todas las bahías generales ocupadas.
Pasos a ejecutar:	1. Se libera una bahía general finalizando su descarga. 2. Se observa a cuál de los tres pedidos se asigna.
Resultado esperado:	1. La bahía se asigna a P2, que registró la llegada más temprana (09:20). 2. El orden de salida de la cola es P2, luego P1 y por último P3. 3. La hora programada de la cita no altera el orden de atención.
Resultado obtenido:	Con tres pedidos en cola (arribos de 40, 80 y 10 minutos de antigüedad), la bahía liberada se asignó al de 80 minutos. La hora programada de la cita no alteró el orden de atención.
Estado:	✔ **PASS**

Caso de Prueba:	**CP-11 — Reprogramación de cita con conservación de historial**
Regla(s) verificada(s):	RN-01, RN-02, RN-14
Descripción:	Validar que al reprogramar una cita quede constancia de la ventana anterior, y que la nueva ventana respete el horario operativo y no se solape con otra cita.
Prerrequisitos:	1. Pedido PROGRAMADO con ventana de 09:00 a 10:00. 2. Otro pedido ya programado de 11:00 a 12:00. 3. Horario operativo de 07:00 a 17:00, de lunes a sábado.
Pasos a ejecutar:	1. Se reprograma el primer pedido a las 11:30 (se solapa con el segundo). 2. Se reprograma a las 18:00 (fuera del horario operativo). 3. Se reprograma a las 14:00, dentro del horario y sin solapamiento.
Resultado esperado:	1. Los dos primeros intentos se rechazan con 400 y el sistema ofrece franjas alternativas disponibles. 2. El tercero prospera: se guarda la nueva ventana y el evento CITA_REPROGRAMADA conserva la ventana anterior junto a la nueva. 3. El historial de la reprogramación queda disponible para auditoría.
Resultado obtenido:	La reprogramación solapada se rechazó con 409 y el sistema ofreció 3 franjas alternativas. La reprogramación fuera de horario se rechazó con 400 y el mensaje «El pedido debe programarse dentro del horario operativo (07:00 a 17:00)». La franja válida respondió 200 y la ventana quedó reprogramada a las 14:00.
Estado:	✔ **PASS**

Caso de Prueba:	**CP-12 — Autogeneración correlativa del número de pedido**
Regla(s) verificada(s):	RN-03
Descripción:	Verificar que el backend genere el número de pedido con el formato del equipo y que el correlativo no se reutilice tras una cancelación.
Prerrequisitos:	1. Parámetro NUMERO_EQUIPO configurado en la colección parametros. 2. Colección de pedidos con el correlativo más alto conocido.

Pasos a ejecutar:	1. Se crea un pedido con POST /api/pedidos. 2. Se cancela ese pedido. 3. Se crea un pedido nuevo y se compara el correlativo.
Resultado esperado:	1. El primer pedido recibe el formato PED-EQUI04-NNNNNNNNN con nueve dígitos. 2. El segundo pedido toma el correlativo siguiente y NO reutiliza el del pedido cancelado. 3. El número se genera en el servidor: no se acepta uno enviado por el cliente.
Resultado obtenido:	El servidor autogeneró PED-EQUI04-000000048 con el formato y los nueve dígitos esperados. Tras cancelar ese pedido, el siguiente recibió PED-EQUI04-000000049: el correlativo avanzó de uno en uno y el del pedido cancelado no se reutilizó.
Estado:	✔️ **PASS**

Estado de ejecución: los doce casos se ejecutaron contra la API desplegada en entorno local el 1 de noviembre de 2026, 19:29 h, con resultado de 12 aprobados de 12. La ejecución está automatizada en el banco de pruebas backend/src/pruebas/, que prepara el escenario de cada caso, lo ejercita contra los endpoints reales y verifica el estado resultante en MongoDB. El campo «Resultado obtenido» de cada ficha transcribe lo que la corrida imprimió, sin agregados. La salida completa se adjunta en el Anexo 6.1.

3.1 Criterios de aceptación en sintaxis Gherkin (BDD)

Los siguientes escenarios expresan los criterios de aceptación de las Historias de Usuario HU-002 y HU-003 en notación Dado / Cuando / Entonces. Los tres primeros provienen literalmente del refinamiento del Product Backlog de la Semana 8 y se conservan sin alterar para no romper la trazabilidad con aquel entregable; los tres restantes se incorporan en esta Semana 12 para cubrir el ciclo de descarga implementado en el Sprint 3.

Escenario 1 — Llegada puntual dentro de la tolerancia · RN-04 · CP-01 · PU-03	
DADO	QUE el pedido tiene cita a las 08:00 AM y la tolerancia configurada es de 15 minutos,
CUANDO	el Operador registra la llegada a las 08:12 AM,
ENTONCES	el sistema guarda la hora real tomada del servidor, asigna la clasificación "A TIEMPO" y deja el pedido disponible para recibir una bahía compatible.

Escenario 2 — Llegada tardía fuera de la tolerancia · RN-04 · CP-03 · PU-04	
DADO	QUE el transporte llega a las 08:25 AM para una cita de las 08:00 AM,
CUANDO	el Operador confirma la llegada,
ENTONCES	el sistema clasifica la llegada como "TARDÍO", registra la diferencia de 25 minutos en la bitácora y mantiene el pedido elegible para descarga.

Escenario 3 — Clasificación automática como ausente · RN-06 · CP-04	
DADO	QUE venció la ventana horaria de una cita y no se registró ninguna llegada,
CUANDO	el sistema ejecuta la verificación automática de ausencias,
ENTONCES	el proveedor queda clasificado como "AUSENTE", la franja vuelve a estar disponible y el evento se guarda en la bitácora para consultarlo después.

Escenario 4 — Producto de construcción asignado al Gateway 5 · RN-08 · CP-05 · PU-06	
DADO	QUE un pedido registrado trae insumos de "construcción" y el Gateway 5 está libre,
CUANDO	el Coordinador inicia la descarga de ese pedido en el Gateway 5,
ENTONCES	el sistema acepta la operación, marca el Gateway 5 como OCUPADO y cambia el pedido a DESCARGANDO.

Escenario 5 — El Gateway 5 no se usa para producto general · RN-08 · CP-06 · PU-07	
DADO	QUE los Gateways 1, 2, 3 y 4 están ocupados y el Gateway 5 está libre,
CUANDO	se intenta iniciar la descarga de un pedido de producto "general" en el Gateway 5,
ENTONCES	el sistema NO asigna el Gateway 5, responde con un error de incompatibilidad de carga y el pedido permanece en espera de una bahía general.

Escenario 6 — Promoción automática al liberarse una bahía · RN-11 · RN-12 · CP-07	
DADO	QUE hay dos pedidos generales esperando y uno de ellos registró su llegada antes que el otro,
CUANDO	se finaliza la descarga que ocupaba una bahía general,
ENTONCES	el sistema libera la bahía y la asigna de inmediato al pedido que lleva más tiempo esperando, sin intervención del Operador.

Lectura de la trazabilidad: cada escenario referencia el caso de prueba funcional que lo ejecuta (CP) y, cuando la regla se puede aislar en una función pura, también la prueba unitaria que la verifica (PU). Esa doble referencia es la que permite recorrer la trazabilidad completa desde el criterio de aceptación escrito por el Product Owner hasta la aserción que corre en Jest.

3.2 Observaciones surgidas de la ejecución

La ejecución de los casos no se limitó a confirmar lo esperado: dejó dos hallazgos que se documentan aquí porque afectan la lectura de los resultados. El primero es una limitación de la API detectada al verificar RN-13; el segundo, un defecto del propio banco de pruebas que impedía ejecutarlo dos veces seguidas. Ambos se consignan con la corrección aplicada.

OBS-01 · CP-09 · RN-13 y RN-09	
Hallazgo	El backend rechaza una bahía FUERA DE SERVICIO con el mismo mensaje genérico que usa para una bahía ocupada: «El gateway no está disponible: ya fue ocupado o no está LIBRE». El comportamiento es correcto —la bahía inhabilitada no recibe descargas— pero la respuesta no permite a quien consume la API distinguir si la bahía está ocupada o fuera de servicio.
Corrección aplicada	La primera versión de CP-09 daba PASS sin probar RN-13: dejaba las otras cuatro bahías ocupadas, de modo que el rechazo podía deberse a la falta de disponibilidad general. Se rediseñó el caso dejándolas LIBRES y agregando un control que confirma que el mismo pedido sí es aceptado en una bahía sana. Se sugiere al equipo de desarrollo diferenciar el mensaje en un sprint posterior.

OBS-02 · Repetibilidad de la suite	
Hallazgo	En la segunda corrida, CP-11 y CP-12 fallaron con 409 por solapamiento de franja horaria. La causa no estaba en el backend sino en la rutina de limpieza del banco de pruebas: esos dos casos crean pedidos a través de POST /api/pedidos, que reciben un correlativo real y no el prefijo con el que la limpieza los identificaba, de modo que quedaban vivos ocupando la franja.
Corrección aplicada	El rechazo con 409 era legítimo: la franja efectivamente estaba ocupada. Se corrigió la limpieza para eliminar también los pedidos asociados a proveedores del banco y se agregó un barrido inicial de pedidos huérfanos, que en la corrida de validación detectó y eliminó 5 registros residuales. La repetibilidad se verificó ejecutando la suite dos veces seguidas sin tocar nada: la segunda corrida reportó 0 huérfanos y CP-12 devolvió los mismos correlativos 000000048 y 000000049 que la primera. Que el correlativo no avanzara es la prueba de que la limpieza devolvió la base a su punto de partida: si hubiera quedado algún pedido vivo, el contador habría seguido desde 000000050.

OBS-03 · PI-02, PI-03 y PI-04 · Dependencia de la hora de ejecución	
Hallazgo	La primera versión de las pruebas integrales programaba las citas «dentro de unos minutos» tomando la hora actual. Al ejecutarlas a las 19:26, POST /api/pedidos las rechazó con 400 porque la ventana caía fuera del horario operativo del depósito, que va de 07:00 a 17:00. El rechazo era correcto: el backend estaba aplicando RN-01 tal como debe.
Corrección aplicada	El defecto estaba en el diseño de la prueba, no en el sistema. Una prueba que solo pasa entre las 07:00 y las 17:00 de un día hábil no verifica el sistema sino el reloj de quien la ejecuta. Se rediseñaron los tres casos para programar las citas en el próximo día hábil a horas distintas, con lo que la suite corre a cualquier hora. El efecto secundario es que la llegada se registra muy por delante de la ventana y el sistema la clasifica como ANTICIPADO: es una de las cuatro clasificaciones válidas de RN-04 y un estado elegible para descarga, de modo que PI-03 sigue ejercitando el ciclo completo. Las dos horas distintas son necesarias porque dos ventanas simultáneas se solaparían y RN-02 rechazaría la segunda.

Criterio del equipo: las dos observaciones se documentan en lugar de corregirse en silencio. OBS-01 describe una prueba que daba PASS sin probar lo que decía probar, y OBS-02, una suite que solo funcionaba la primera vez que se ejecutaba. Son la clase de defecto que vuelve inútil a un plan de QA: una prueba que pasa por el motivo equivocado da una garantía falsa, y es más peligrosa que una que falla de frente.

4. PLAN DE PRUEBAS UNITARIAS (BACKEND API Y LÓGICA DE NEGOCIO)

Las pruebas unitarias se ejecutan con Jest sobre TypeScript mediante ts-jest, y validan de forma aislada las funciones que concentran la lógica de negocio del backend Node.js/Express. Ninguna de ellas levanta MongoDB ni el servidor HTTP: los servicios reciben la configuración operativa por parámetro, y en el único caso que consulta la base —la generación del correlativo de pedido— el modelo se sustituye por un doble de prueba.

4.1 Fichas de pruebas unitarias ejecutadas

ID	Archivo en src/__tests__/	Función evaluada	Datos de entrada	Salida esperada (assert)	PASS
PU-01	`puntualidad`	`minutosDeDiferencia()`	llegada 09:12, inicio de ventana 09:00	`12 (entero, redondeado al minuto)`	Sí
PU-02	`puntualidad`	`clasificarLlegada()`	diferencia −16 min, tolerancia 15	`'ANTICIPADO'`	Sí
PU-03	`puntualidad`	`clasificarLlegada()`	diferencia +12 min, tolerancia 15	`'A TIEMPO'`	Sí
PU-04	`puntualidad`	`clasificarLlegada()`	diferencia +25 min, tolerancia 15	`'TARDÍO'`	Sí
PU-05	`puntualidad`	`clasificarLlegada()` `limiteDeAusencia()`	diferencia +61 min, límite 60	`'AUSENTE'; las 4 clasificaciones cubiertas`	Sí
PU-06	`gateways`	`tipoCargaEsperado()`	numeroGateway = 5	`'construcción'`	Sí
PU-07	`gateways`	`tipoCargaEsperado()`	numeroGateway = 1, 2, 3 y 4	`'general' en los cuatro casos`	Sí
PU-08	`gateways`	`Catálogo de bahías`	rango 1 a 5	`Las 5 bahías con tipo de carga asignado`	Sí
PU-09	`gateways`	`TIPOS_CARGA ESTADOS_GATEWAY`	constantes del modelo	`['construcción','general']; ['LIBRE','OCUPADO','FUERA DE SERVICIO']`	Sí
PU-10	`ventanaHoraria`	`esDiaHabil()` `limitesDelDia()`	jueves y domingo; horario 07-17	`true / false; apertura 07:00 y cierre 17:00`	Sí
PU-11	`ventanaHoraria`	`estaDentroDelHorario()`	ventana 16:30 + 60 min	`false (termina 17:30, pasa el cierre)`	Sí
PU-12	`ventanaHoraria`	`seSolapan()`	09:00-10:00 contra 10:00-11:00	`false (turnos consecutivos son válidos)`	Sí
PU-13	`ventanaHoraria`	`buscarSiguienteHueco()`	desde 09:00, ocupada 09:00-10:00	`Hueco a las 10:00`	Sí
PU-14	`ventanaHoraria`	`buscarAlternativas()`	sábado 16:30, duración 60 min	`3 alternativas que saltean el domingo`	Sí
PU-15	`numeroPedido`	`generarNumeroPedido()`	sin pedidos previos, equipo "04"	`'PED-EQUI04-000000001'`	Sí
PU-16	`numeroPedido`	`generarNumeroPedido()`	último 'PED-EQUI04-	`'PED-EQUI04-000000148'`	Sí

ID	Archivo en src/__tests__/	Función evaluada	Datos de entrada	Salida esperada (assert)	PASS
			000000147'		
PU-17	numeroPedido	generarNumeroPedido()	equipo "01"; filtro por prefijo	'PED-EQUI01-000000001'; regex '^PED-EQUI04-'	Sí
PU-18	numeroPedido	generarNumeroPedido()	correlativo no numérico	Reinicia en 'PED-EQUI04-000000001'	Sí
PU-19	login	decidirPasoDeLogin()	los cuatro cruces de MFA exigido y MFA configurado	ACCESO_DIRECTO, CONFIGURAR o VERIFICAR según corresponda	Sí
PU-20	login	decidirPasoDeLogin()	barrido de los cuatro casos	Nunca ACCESO_DIRECTO con MFA exigido ni con MFA ya configurado	Sí
PU-21	mfa	esFormatoDeCodigoValido()	'012345'; '12 34 56'	true (acepta ceros a la izquierda y espacios de copiar y pegar)	Sí
PU-22	mfa	esFormatoDeCodigoValido()	'12345'; 'abcdef'; null	false en los tres casos	Sí
PU-23	mfa	generarCodigosDeRecuperacion()	cantidad declarada por el modelo	Formato XXXXX-XXXXX, sin repetidos ni caracteres confundibles	Sí
PU-24	mfa	hashearCodigosDeRecuperacion()	tanda de códigos de recuperación	Ninguno queda en texto plano y siguen siendo verificables	Sí
PU-25	mfa	intentosMfa	cinco códigos incorrectos sobre un mismo token temporal	Corta al sexto; el conteo es por token y un acierto lo reinicia	Sí

4.2 Cobertura de código sobre los módulos bajo prueba

Módulo	Reglas que implementa	% Sentencias	% Ramas	% Funciones
services/puntualidad.ts	RN-04	100 %	100 %	100 %
services/numeroPedido.ts	RN-03	100 %	100 %	100 %
services/ventanaHoraria.ts	RN-01, RN-02, RN-14	92.64 %	88.88 %	83.33 %
models/Gateway.ts	RN-07, RN-08	75.00 %	33.33 %	50.00 %
services/login.ts	RN-15	100 %	100 %	100 %
services/mfa.ts	RN-16	94.23 %	91.66 %	76.92 %
services/intentosMfa.ts	RN-17	100 %	100 %	100 %

Alcance de la medición: la cobertura reportada corresponde a los módulos incluidos en el alcance de esta suite. La cobertura global del proyecto es menor porque el reporte contabiliza también los controladores, las rutas y los servicios de KPIs y auditoría, que se validan en los niveles funcional e integral y no en el unitario. Declarar aquí la cifra global sería atribuir a las pruebas unitarias un alcance que no tienen.

4.3 Evidencia de ejecución

La suite completa se ejecuta con el comando npm test desde el directorio backend/. El resultado consolidado de la última corrida del equipo es el siguiente; la salida íntegra, caso por caso, se adjunta en el Anexo 6.1.

Métrica	Resultado
Suites de prueba	6 aprobadas de 6 totales
Casos de prueba	78 aprobados de 78 totales
Casos fallidos	0
Tiempo de ejecución	10.426 s
Entorno	Node.js 22.22.0 · Jest 29.7.0 · ts-jest 29.2.5 · Windows

5. PLAN DE PRUEBAS INTEGRALES (FULLSTACK)

Las pruebas integrales evalúan la interacción conjunta de la interfaz reactiva (Next.js con App Router y Bootstrap), la API REST (Node.js/Express con TypeScript), la persistencia en MongoDB y el control de acceso basado en roles. A diferencia de las unitarias, aquí el estado compartido es parte de lo que se verifica.

5.1 Flujos de integración evaluados

ID	Componentes	Flujo evaluado	Resultado esperado	Estado de ejecución
PI-01	Next.js ↔ Express ↔ MongoDB	Alta de proveedor y programación de una cita con ventana horaria válida, en su totalidad por los endpoints REST.	El pedido se persiste con el correlativo autogenerado, la ventana calculada por el servidor a partir de la duración estimada, y el registro de auditoría del usuario que lo creó.	EJECUTADO — PASS
PI-02	Garita ↔ Express ↔ MongoDB	Registro de llegada con clasificación automática de puntualidad y asiento en la bitácora de eventos.	La llegada se guarda con la hora fijada por el servidor durante la petición, la clasificación corresponde a la diferencia contra el inicio de la ventana y el hecho queda consultable en GET /api/eventos.	EJECUTADO — PASS
PI-03	Tablero de gateways ↔ Express ↔ MongoDB	Ciclo transaccional completo: alta, programación, llegada, inicio de descarga, cierre y promoción automática del siguiente de la cola.	La bahía pasa a OCUPADO y vuelve a LIBRE al finalizar; el pedido descargado queda FINALIZADO y el que esperaba es promovido sin intervención del operador.	EJECUTADO — PASS
PI-04	Dashboard de KPIs ↔ Express ↔ MongoDB	Cálculo de los indicadores sobre la operación registrada, comparando el tablero antes y después de dos arribos.	El tablero devuelve los cinco bloques de indicadores, el cumplimiento contabiliza exactamente los arribos registrados y la distribución por puntualidad es internamente consistente.	EJECUTADO — PASS
PI-05	Login ↔ verificarToken ↔ autorizarRoles ↔ Express	Acceso a los endpoints protegidos con tokens de los tres roles y sin token.	Cada rol alcanza únicamente los endpoints que le corresponden; los demás responden 403 sin filtrar información del recurso, y sin token responden 401.	EJECUTADO — PASS

Resultado obtenido en la corrida

ID	Resultado obtenido
PI-01	El proveedor se dio de alta mediante POST /api/proveedores y la cita se programó con POST /api/pedidos. El pedido quedó persistido con el correlativo PED-EQUI04-000000048, la ventana calculada por el servidor a partir de la duración estimada, y la auditoría a nombre de Carlos Coordinador.
PI-02	La llegada se registró con 200 y el sistema la clasificó como ANTICIPADO, con −870 minutos respecto del inicio de la ventana. El arribo quedó asentado por el servidor a las 2026-11-02T01:29:51.813Z y el hecho disponible en GET /api/eventos para auditoría.
PI-03	El ciclo completo se verificó de punta a punta: alta de proveedor, programación de cita, registro de llegada, inicio de descarga, cierre y promoción automática del siguiente de la cola. La promoción ocurrió sin intervención del operador, en la misma transacción del cierre.
PI-04	El tablero devolvió los cinco bloques de indicadores con el período solicitado. El cumplimiento pasó de 0 a 2 tras registrar dos arribos, es decir que contabilizó exactamente los que se registraron, y la distribución por puntualidad resultó consistente: 2

ID	Resultado obtenido
	pedidos y 100 % acumulado.
PI-05	Los tres roles autenticaron correctamente. Los 13 endpoints protegidos responden 401 sin token. La matriz completa se verificó sobre 13 endpoints y 3 roles: 21 combinaciones permitidas y 18 denegadas con 403. El Operador no alcanza ningún endpoint de administración ni de indicadores, el Administrador no registra llegadas ni inicia descargas, y la administración de usuarios solo la alcanza el Administrador.

Alcance de la cobertura integral: los cinco flujos integrales se ejecutaron de forma automatizada y aprobaron. A diferencia de los casos funcionales CP, estas pruebas no escriben en la base para preparar su escenario: todo ocurre por los endpoints REST, igual que lo haría una persona desde la aplicación. Esa es la distinción entre ambos niveles y lo que las sostiene como End-to-End. La capa de interfaz Next.js no se cubre con pruebas automatizadas sino con el protocolo UAT del Plan de Implementación: validar que la pantalla sirve es algo que corresponde al usuario final, no a un script que simule clics.

5.2 Nota técnica: estrategia de cola y alcance de EDA 0

El Backlog del proyecto contempla una cola de prioridad implementada como Heap binario con política de envejecimiento y complejidad O(log n) en inserción y extracción. Ese componente corresponde al entregable de la asignatura EDA 0 y el atributo de calidad RNF-08 de la Historia de Usuario HU-003, refinada en la Semana 8, declara que se desarrolla en C++ con punteros y plantillas.

La verificación realizada para esta revisión determinó que el componente está implementado en Python utilizando el módulo heapq de la biblioteca estándar. La estructura es efectivamente un heap binario y la política de envejecimiento está implementada y verificada —mejora un nivel de prioridad por cada 20 minutos de espera, con tope en la prioridad máxima— pero el lenguaje y la naturaleza del componente difieren de lo declarado en el SRS. El equipo deja constancia de la discrepancia en lugar de repetir el dato anterior, y propone actualizar RNF-08 en la próxima revisión del Backlog para que describa el componente que realmente existe.

En la Versión Beta Funcional que documenta este plan, el backend de PON 0 resuelve el orden de atención de la cola mediante una consulta ordenada por hora de llegada real, delegando el ordenamiento en el motor de la base de datos. El criterio es el mismo que persigue la política de envejecimiento —atender primero a quien lleva más tiempo esperando en el patio— pero la estructura de datos que lo implementa todavía no es el Heap de EDA 0.

Por esa razón este documento no consigna pruebas de volumen ni mediciones de complejidad algorítmica sobre la cola: medir un O(log n) que el código de esta rama no ejecuta sería reportar un resultado que la implementación no respalda. La regla RN-12 se verifica aquí por su comportamiento observable (CP-10), y la verificación de la eficiencia algorítmica queda planificada para el sprint en que se integre el motor de EDA 0 al backend.

Criterio del equipo: esta decisión se documenta de forma explícita porque afecta la lectura de la matriz de la sección 2.1: RN-12 aparece como verificada en su comportamiento funcional, no en su característica de eficiencia. La trazabilidad con el entregable de EDA 0 se mantiene abierta y se cerrará cuando ambos componentes se integren.

6. ANEXOS Y DECLARACIÓN DE GOBERNANZA SCM

6.1 Anexo: salidas completas de las suites de prueba

Se adjuntan como anexos del entregable los archivos de texto con la salida íntegra de las dos suites. El primero corresponde a npm test, con las 78 aserciones unitarias, su resultado individual y su tiempo de ejecución. El segundo corresponde a npm run test:funcional, con los doce casos CP-01 a CP-12 y las cinco pruebas integrales PI-01 a PI-05, las verificaciones de cada uno y el resumen final. Ambos archivos se versionan junto con las clases de prueba en la rama de QA del repositorio.

Suite	Comando	Resultado	Anexo
Unitarias	`npm test`	78 de 78 aprobadas	`salida-jest.txt`
Funcionales e integrales	`npm run test:funcional`	17 de 17 aprobados	`salida-funcionales.txt`

6.2 Repositorio oficial y artefactos de QA

Artefacto	Referencia
Repositorio oficial del equipo	`https://github.com/202401579-hue/Proyecto-Integrador`
Rama de integración de pruebas	`feature/HU-03-pruebas-qa` (integrada a develop)
Pull Request con Code Review	`https://github.com/202401579-hue/Proyecto-Integrador/pull/21` Aprobado por Victor5977 el 19/10/2026
Etiqueta de liberación Beta	`v2.1.0-beta` `https://github.com/202401579-hue/Proyecto-Integrador/releases/tag/v2.1.0-beta`
Clases de prueba unitarias	`backend/src/__tests__/` (6 archivos, 78 casos)
Banco de pruebas funcionales	`backend/src/pruebas/` (8 archivos, 17 casos)
Configuración del runner	`backend/jest.config.js` · `backend/tsconfig.test.json`

6.3 Definition of Done del incremento

#	Criterio de la Definition of Done	Estado
1	Las reglas de negocio RN-01 a RN-17 están catalogadas y trazadas a un endpoint o componente real del backend.	✔️ Cumplido
2	Existe una suite de pruebas unitarias automatizada y ejecutable con un solo comando (npm test).	✔️ Cumplido
3	La suite unitaria se ejecuta sin fallos: 78 de 78 casos aprobados.	✔️ Cumplido
4	Los criterios de aceptación están expresados en sintaxis Gherkin y vinculados a sus casos de prueba.	✔️ Cumplido
5	La incorporación de las pruebas no rompe la compilación del proyecto	✔️ Cumplido

#	Criterio de la Definition of Done	Estado
	(npm run build).	
6	Los doce casos de prueba funcionales están diseñados, trazados y ejecutados, con su resultado consignado.	✔️ Cumplido
6b	La suite funcional es repetible: limpia su propio estado y puede ejecutarse varias veces sin intervención manual.	✔️ Cumplido
7	El Pull Request de QA cuenta con Code Review aprobado por un integrante distinto del autor.	✔️ Cumplido
8	La etiqueta de liberación v2.1.0-beta está creada y apunta al commit de cierre del incremento.	✔️ Cumplido

6.4 Declaración de uso de Inteligencia Artificial (estándar FEEN)

📌 **DECLARACIÓN OBLIGATORIA:** para la elaboración de este Plan de Verificación y Validación, el equipo utilizó las herramientas de Inteligencia Artificial Claude y ChatGPT con tres propósitos acotados: (1) guiar mediante preguntas el razonamiento sobre qué niveles de prueba corresponden a cada tipo de regla de negocio; (2) apoyar la redacción y la ortografía del documento; y (3) asistir en la escritura de las aserciones de la suite Jest a partir del código fuente que el propio equipo desarrolló. La selección de las reglas a verificar, el diseño de los casos de prueba, la decisión de no reportar métricas de eficiencia algorítmica sobre un componente aún no integrado y la ejecución y validación de los resultados fueron realizadas y verificadas críticamente por los integrantes del Equipo Scrum, que asumen la responsabilidad técnica completa de lo aquí consignado, de conformidad con los Lineamientos del Uso de la Inteligencia Artificial de la FEEN / UJMD.

Prompts y consultas ejecutadas

Propósito de la consulta	Resultado y validación humana
Contrastar qué reglas de negocio pueden verificarse en pruebas unitarias y cuáles requieren niveles funcional o integral.	Se adoptó el criterio de aislar únicamente las funciones puras. El equipo revisó el código fuente para confirmar qué servicios lo eran.
Apoyo en la redacción de las aserciones Jest a partir de los servicios del backend.	Las pruebas se ejecutaron en el entorno local del equipo y se verificó que los 78 casos pasaran y que npm run build siguiera compilando.
Revisión de la coherencia entre el catálogo RN-01 a RN-17, los endpoints y los casos de prueba.	El equipo verificó endpoint por endpoint contra el código de las rutas Express antes de consignarlos en la matriz.